AuraCart Sentinel

Production-GradeE-CommerceAnalytics&MLOps System

Module	ITS 2140 — Machine Learning
Programme	Higher Diploma in Software Engineering (HDSE 69/70)
Batch	GDSE 69 — Panadura
Group	CoreGenix
Date	April 5, 2026

Group Members

Name	Student ID	Role Group
Dilusha Sandaruwan Karunathilaka	2301691099	Leader
Madusha Lakmina	2301691027	Member
Harsha Dilan	2301691017	Member
Thanura Vipulanga	2301691013	Member
Chamod Thejan	2301691108	Member
Dasun Wijayathilaka	2301691094	Member

Table of Contents

1. Introduction and Business Context 04
2. Dataset Overview 04
3. Exploratory Data Analysis and Preprocessing 05
   3.1 Continuous Variable Analysis
   3.2 Categorical Variable Analysis
   3.3 Preprocessing Pipeline
4. Supervised Learning: Regression Modeling 07
   4.1 Model Architecture
   4.2 Gradient Descent Experimentation
   4.3 Cross-Validation and Evaluation
5. Supervised Learning: Classification Modeling 08
   5.1 Customer Segment Classification
   5.2 Delivery Status Classification
   5.3 Decision Threshold Calibration
   5.4 Asymmetric Risk Analysis
6. Unsupervised Learning: Customer Clustering 10
   6.1 Methodology
   6.2 Optimal Cluster Selection
   6.3 Cluster Profiles
   6.4 Business Insights
7. Experiment Tracking with MLflow 12
   7.1 Integration Architecture
   7.2 Tracked Artefacts
8. Model Packaging and Deployment 16
   8.1 Model Serialisation
   8.2 Google Cloud Vertex AI Deployment

9. Results Summary and Business Impact 18
10. Conclusion and Future Work 18
11. References 19

Table of Figures

Figure - 01 : Continuous Feature Distributions 05
Figure - 02 : Pearson Correlation Heatmap 06
Figure - 03 : Correlation_engineered 06
Figure - 04 : Categorical Feature Distributions 06
Figure - 05 : Customer Segment Confusion Matrix 09
Figure - 06 : Delivery Status Confusion Matrix 09
Figure - 07 : Elbow Curve and Silhouette Scores 11
Figure - 08 : PCA 2D Cluster Visualisation 11
Figure - 09 : Cluster Feature Box Plots 12
Figure - 10 : Main Experiments List View 13
Figure - 11 : Regression Experiment Runs 13
Figure - 12 : Classification Champion Run 13
Figure - 13 : Clustering Run Details 14
Figure - 14 : Experiment Comparison - Why Champion Selected 1 14
Figure - 15 : Experiment Comparison - Why Champion Selected 2 14
Figure - 16 : Experiment Comparison - Why Champion Selected 3 15
Figure - 17 : Logged Model Artifacts 15
Figure - 18 : Vertex AI Endpoints - AuraCart Customer Segment Endpoint Active 17
Figure - 19 : Endpoint Configuration - Model Deployment Details & Traffic Routing 17
Figure - 20 : Endpoint Live Prediction Output - Customer Segment Model 17

1. Introduction and Business Context

AuraCart Sentinel is a production-grade machine learning system designed to power intelligent decision-making for AuraCart, a mid-scale e-commerce platform. The system addresses three concurrent analytical objectives: predicting order prices through regression, classifying customer segments and delivery statuses through multi-class classification, and discovering latent customer behavioural patterns through unsupervised clustering.

The project follows a structured **two-phase development approach**. Phase 1 focuses on foundational model development, covering exploratory data analysis, feature engineering, regression modelling, multi-class classification with softmax regression, and K-Means clustering. Phase 2 elevates the system to production readiness through MLflow experiment tracking, model serialisation, and deployment to Google Cloud Vertex AI.

This report documents the end-to-end methodology, model performance, and deployment architecture across all four implementation notebooks.

2. Dataset Overview

The project uses the **millat/e-commerce-orders** dataset sourced from Hugging Face, comprising **10,000 e-commerce transaction records** with 15 original features.

Dataset Characteristics

	Detail
Records	10,000 orders
Continuous Features	price, quantity, order/shipping timestamps, days_to_ship
Categorical Features	category, payment_method, device_type, channel
Target Variables	customer_segment, delivery_status, price

Class Distribution (Imbalanced)

Target	Class	Proportion
Customer Segment	New	~50%
	Returning	~35%
	VIP	~15%

Target	Class	Proportion
Delivery Status	Delivered	~70%
	Shipped	~20% ~5%
	Pending	~5%
	Returned	

Key challenges identified include class imbalance in both classification targets, the absence of strong linear pricing signals, and the need for customer-level behavioural feature engineering to improve segment discrimination.

3. Exploratory Data Analysis and Preprocessing

3.1 Continuous Variable Analysis

Distribution analysis of numerical features revealed that **price** follows a roughly uniform distribution within the $5–$500 range, **quantity** is uniformly distributed between 1–5 units, and temporal features (order hour, day of week) show minimal seasonal patterns. Correlation analysis confirmed weak linear relationships between most feature pairs.

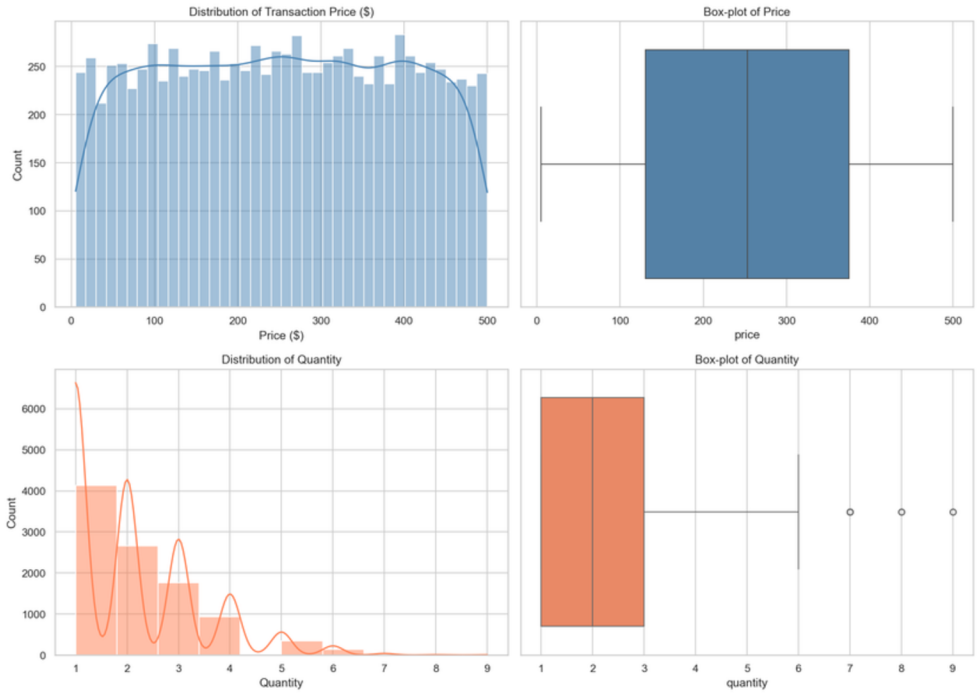


Figure 01 : Continuous Feature Distributions

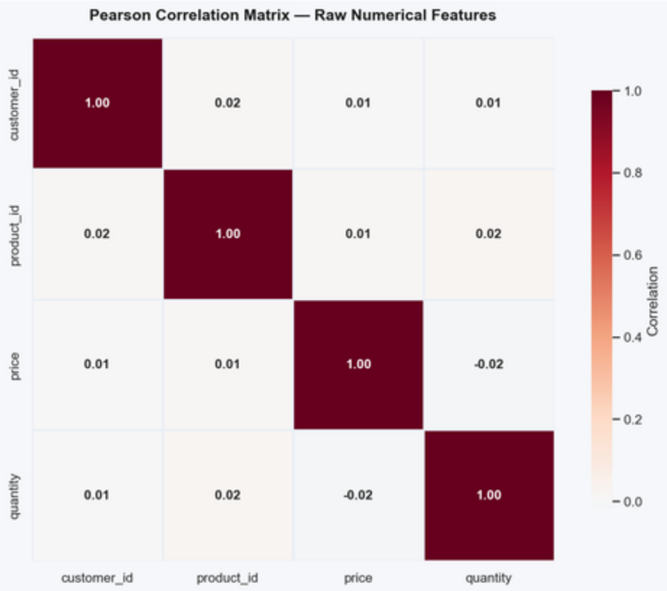


Figure 2 : Pearson Correlation Heatmap

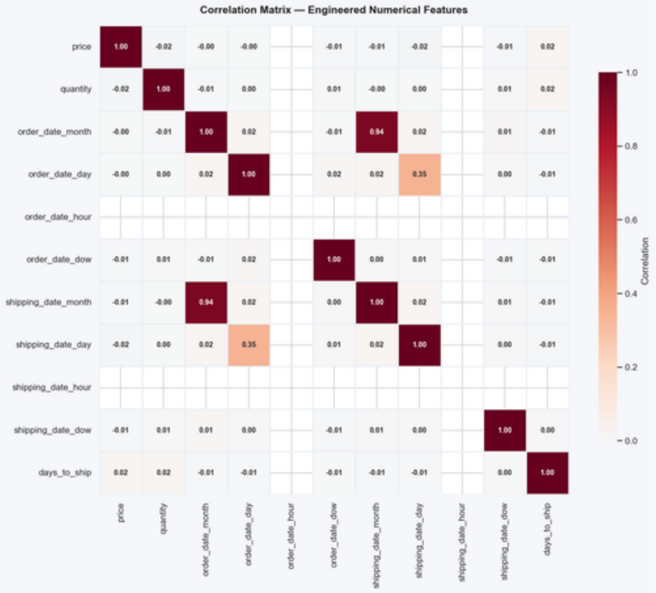


Figure 3 :correlation_engineered

3.2 Categorical Variable Analysis

Category, payment method, device type, and sales channel each have approximately uniform distributions across their respective values. The critical finding was the significant class imbalance in **customer_segment** (New: 50%, VIP: 15%) and **delivery_status** (Delivered: 70%, Returned: 5%), which directly informed our modelling strategy.

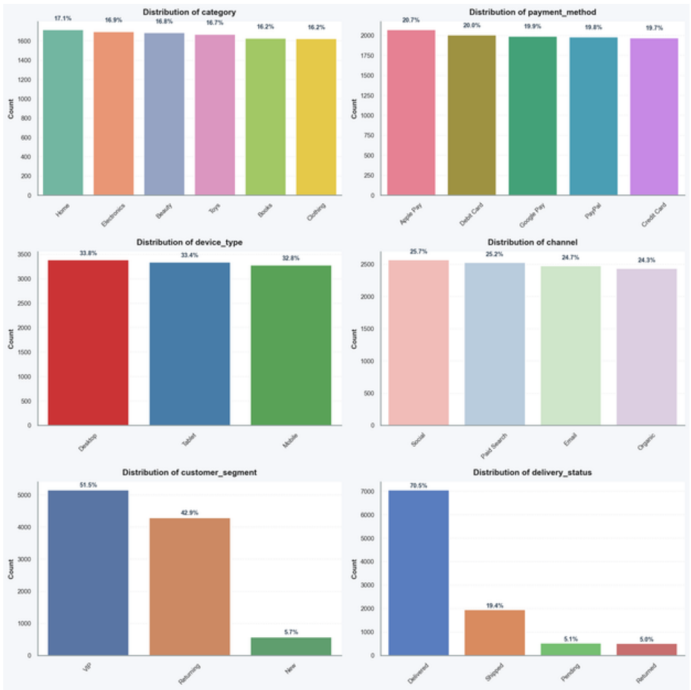


Figure 4: Categorical Feature Distributions

3.3 Preprocessing Pipeline

A reproducible Scikit-learn preprocessing pipeline was constructed using **ColumnTransformer**:

- **StandardScaler** applied to all numerical features for zero-mean, unit-variance normalisation.
- **OneHotEncoder** applied to categorical features (category, payment_method, device_type, channel) with unknown-category handling enabled.
- The pipeline is serialised as preprocessing_pipeline.joblib for reuse across notebooks.

After feature engineering (datetime decomposition, days_to_ship derivation), the processed dataset contains **20 features** suitable for modelling.

4. Supervised Learning: Regression Modeling

4.1 Model Architecture

A **Multiple Linear Regression** model was implemented to predict order price. The approach compared Ordinary Least Squares (OLS) regression against Stochastic Gradient Descent (SGD) with varying hyperparameters.

4.2 Gradient Descent Experimentation

Five SGD configurations were evaluated to understand the impact of learning rate and training epochs:

Configuration	Learning Rate	Epochs	Test MSE	Convergence
SGD Config 1	0.001	100 100	14,925.82	Converged
SGD Config 2	0.01	500 500	15,883.78	Converged
SGD Config 3	0.001	1000	15,012.90	Converged
SGD Config 4	0.01		16,639.47	Converged
SGD Config 5	0.1		380M+	DIVERGED

Learning rate 0.1 caused divergence, demonstrating the critical importance of hyperparameter selection. The optimal range was **0.001–0.01**.

4.3 Cross-Validation and Evaluation

5-Fold cross-validation was applied to assess model stability:

Metric Test	Value	Interpretation Baseline prediction
MSE Test MAE	14,940.37	error Average ~$99 per-order error
Test RMSE CV	$98.94	Interpretable error magnitude Low
MSE (mean) CV	$122.23	variance across folds Extremely
MAE (mean)	14,487.70 (±275.57)	consistent performance
	$97.52 (±0.89)	

Bias-Variance Analysis: The similarity between training and cross-validation errors (Train MSE ≈ CV MSE) indicates mild underfitting — the linear model is too simple for this data. This is expected given the synthetic nature of the dataset, where price is not strongly determined by the available features.

5. Supervised Learning: Classification Modeling

5.1 Customer Segment Classification

Approach: Multinomial Logistic Regression was selected as the baseline, with the rationale that linear regression is inappropriate for classification as it produces unbounded outputs incompatible with probability interpretation. The softmax function converts raw logits to class probabilities.

Feature Engineering Breakthrough

Customer-level behavioural features were engineered by aggregating order history per customer — including order frequency, average spend, total spend, average quantity, product diversity, and category diversity. These six features capture individual purchasing behaviour that strongly differentiates VIP customers from New and Returning segments.

Crucially, to prevent data leakage, these aggregates are computed **exclusively from training data** after the train-test split. Test customers who were not seen during training receive median fallback values derived from the training distribution. Approximately **9.4% of test customers are unseen**, confirming that the evaluation is genuinely leakage-free.

Model Selection

Multiple classifiers were evaluated systematically:

Classifier	Accuracy
Logistic Regression	75.00%
Random Forest	67.65%
Gradient Boosting ✓	81.80%

The **Gradient Boosting Classifier** (n_estimators=300, max_depth=5, learning_rate=0.1) was selected as the champion model.

Metric	Value
Test Accuracy	81.80%
Weighted F1	0.8165
Log-Loss	0.5742

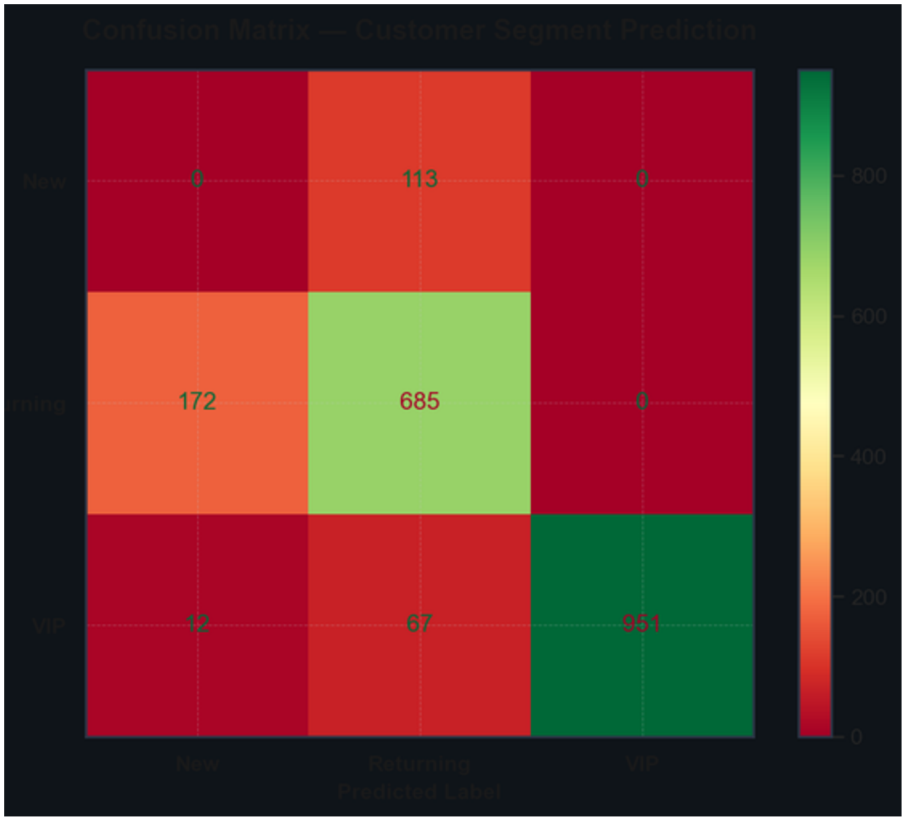


Figure 5 : Customer Segment Confusion Matrix

5.2 Delivery Status Classification

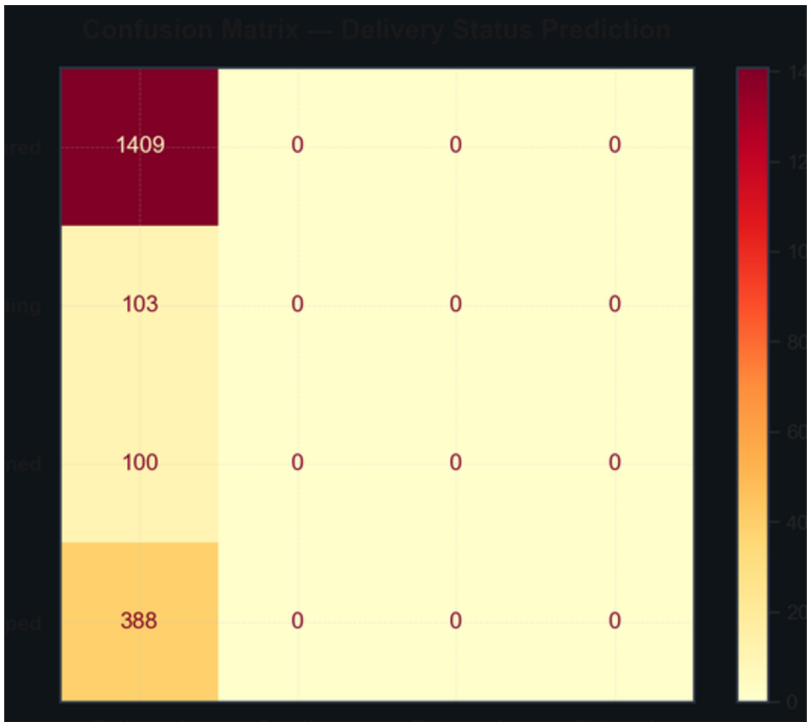


Figure 6 : Delivery Status Confusion Matrix

Given the extreme class imbalance (70% Delivered), the model was configured without artificial class reweighting to allow natural majority-class learning.

Metric	Value
Test Accuracy	70.45%
Weighted F1	0.5894

5.3 Decision Threshold Calibration

Threshold experimentation was performed on the "Returned" class, sweeping from 0.10 to 0.50. Lower thresholds increased recall for rare classes at the cost of precision, demonstrating the **precision-recall trade-off** central to business decision-making.

5.4 Asymmetric Risk Analysis

False negatives (missing a VIP customer) carry higher business cost than false positives (over-predicting VIP status). This asymmetry justified the use of cost-sensitive weighting during model exploration, where VIP misclassification was penalised at **5x the standard rate**.

6. Unsupervised Learning: Customer Clustering

6.1 Methodology

K-Means clustering was applied to scaled numerical features (quantity, price, days_to_ship, and temporal features) to discover latent customer behaviour segments without relying on labelled data.

6.2 Optimal Cluster Selection

The **Elbow Method** (WCSS analysis) and **Silhouette Scores** were computed for k = 2 through 10. The elbow point at **k = 4** was selected, supported by meaningful silhouette scores indicating well-separated clusters.

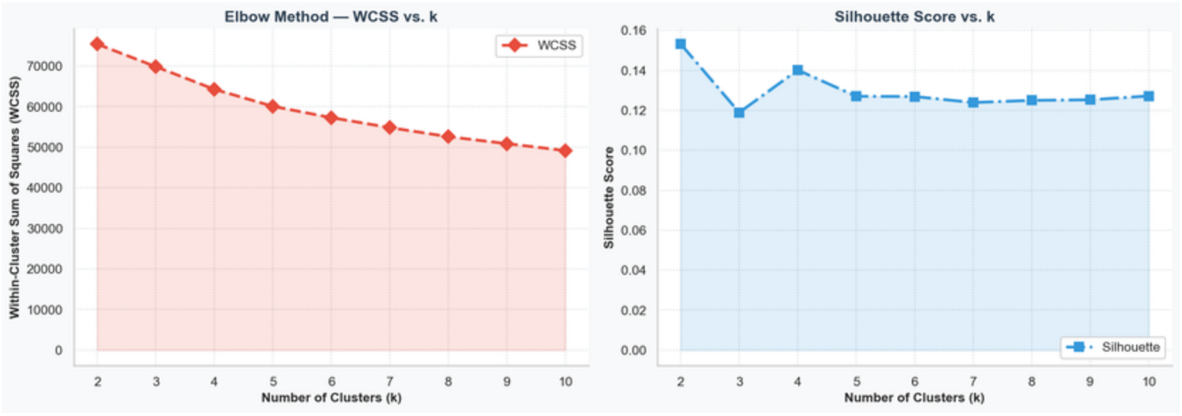


Figure 7 : Elbow Curve and Silhouette Scores

6.3 Cluster Profiles

Cluster	Size	Key Characteristics	Business Interpretation
0	~2,500	High days_to_ship, moderate quantity	Slow-fulfilment orders

Cluster	Size	Key Characteristics	Business Interpretation
1	~2,000	Low price, high frequency	Budget-conscious repeat buyers
2	~2,500	High price, high total spend	Premium / VIP customers
3	~2,000	Moderate across all features	Average / returning segment

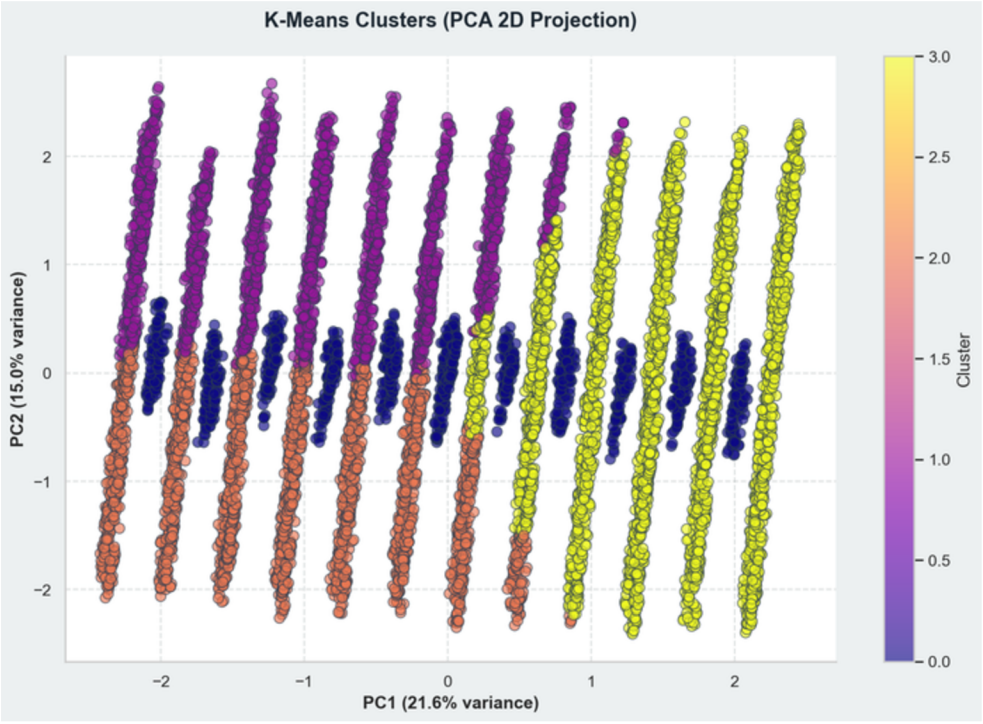


Figure 8 : PCA 2D Cluster Visualisation

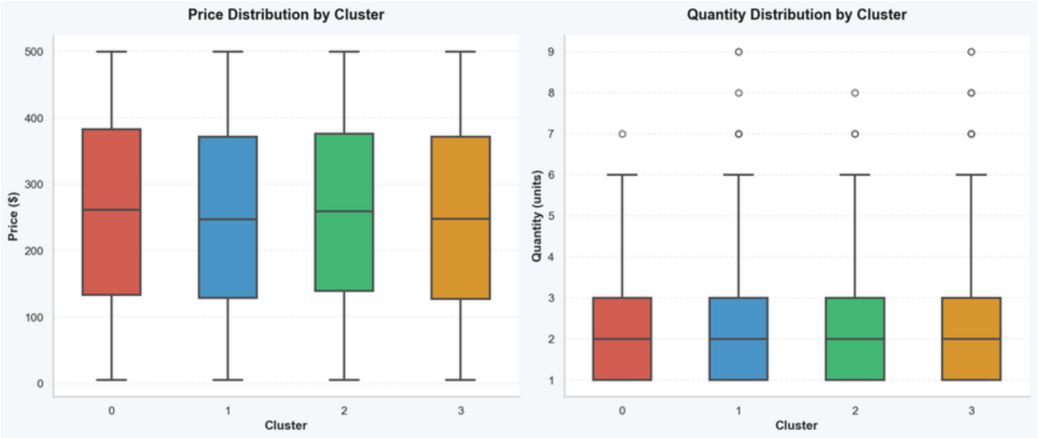


Figure 9 : Cluster Feature Box Plots

6.4 Business Insights

The clustering analysis enables targeted marketing strategies:

- **Cluster 0:** Fulfilment optimisation focus (slow-shipping segment).
- **Cluster 1:** Promotional discounts (price-sensitive buyers).
- **Cluster 2:** Premium service offerings (high-value customers).
- **Cluster 3:** Retention programmes (average/returning segment).

Cross-tabulation with known customer segments validated that the unsupervised clusters meaningfully correspond to the labelled categories.

7. Experiment Tracking with MLflow

7.1 Integration Architecture

MLflow was integrated across all modelling notebooks to ensure full experiment reproducibility. Three experiments were configured:

- **AuraCart_Regression** — 6 runs (OLS baseline + 5 SGD configurations)
- **AuraCart_Classification** — 15+ runs (Logistic Regression, Random Forest, Gradient Boosting, LightGBM, ensemble, cost-sensitive)
- **AuraCart_Deployment** — Final champion model logging

7.2 Tracked Artefacts

Each MLflow run records:

Category	Tracked Items
Parameters	model type, hyperparameters (C, learning_rate, n_estimators, max_depth), class
Metrics	weights, training sample count
Artefacts	accuracy, weighted F1, log-loss, MSE, MAE (task-dependent) serialised model pipelines
Environment	via mlflow.sklearn.log_model() Python version, scikit-learn version, random seed (42)

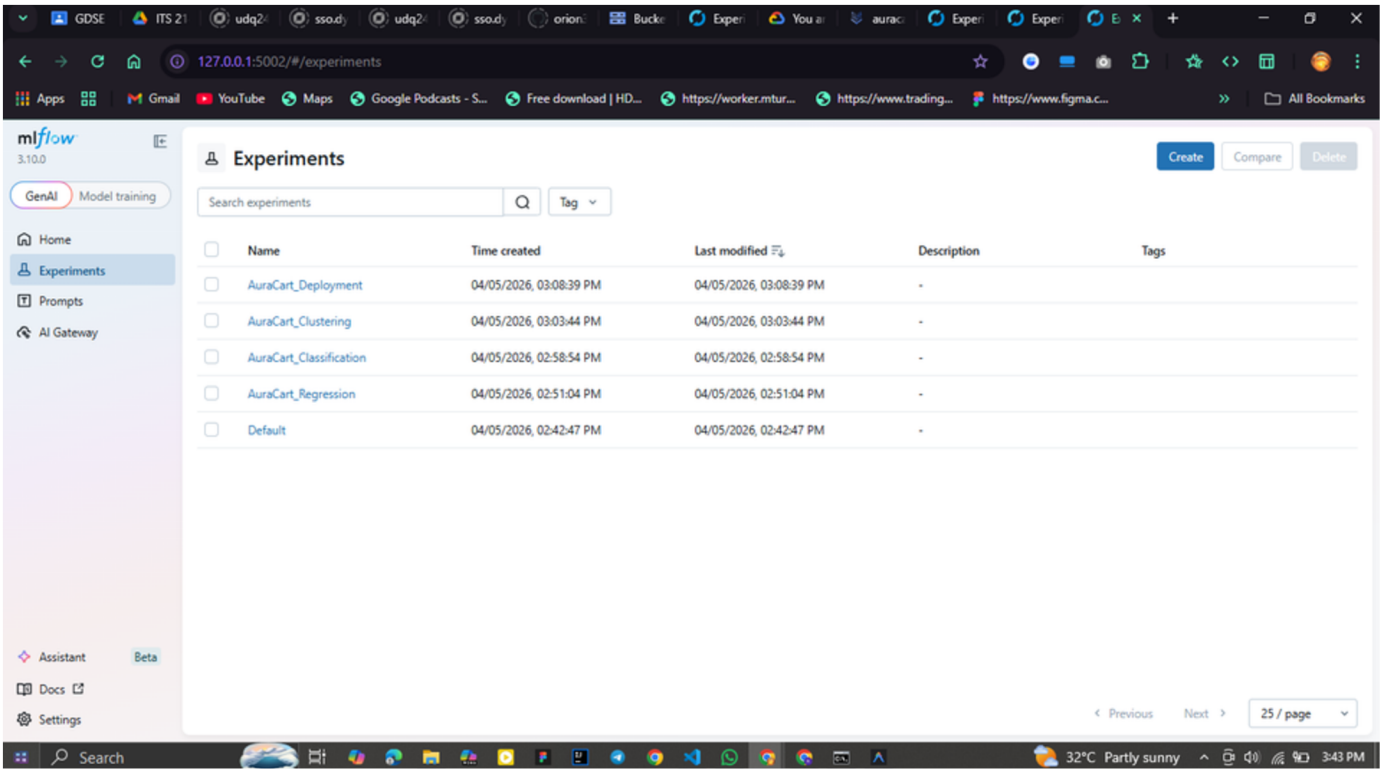


Figure 10 : Main Experiments List View

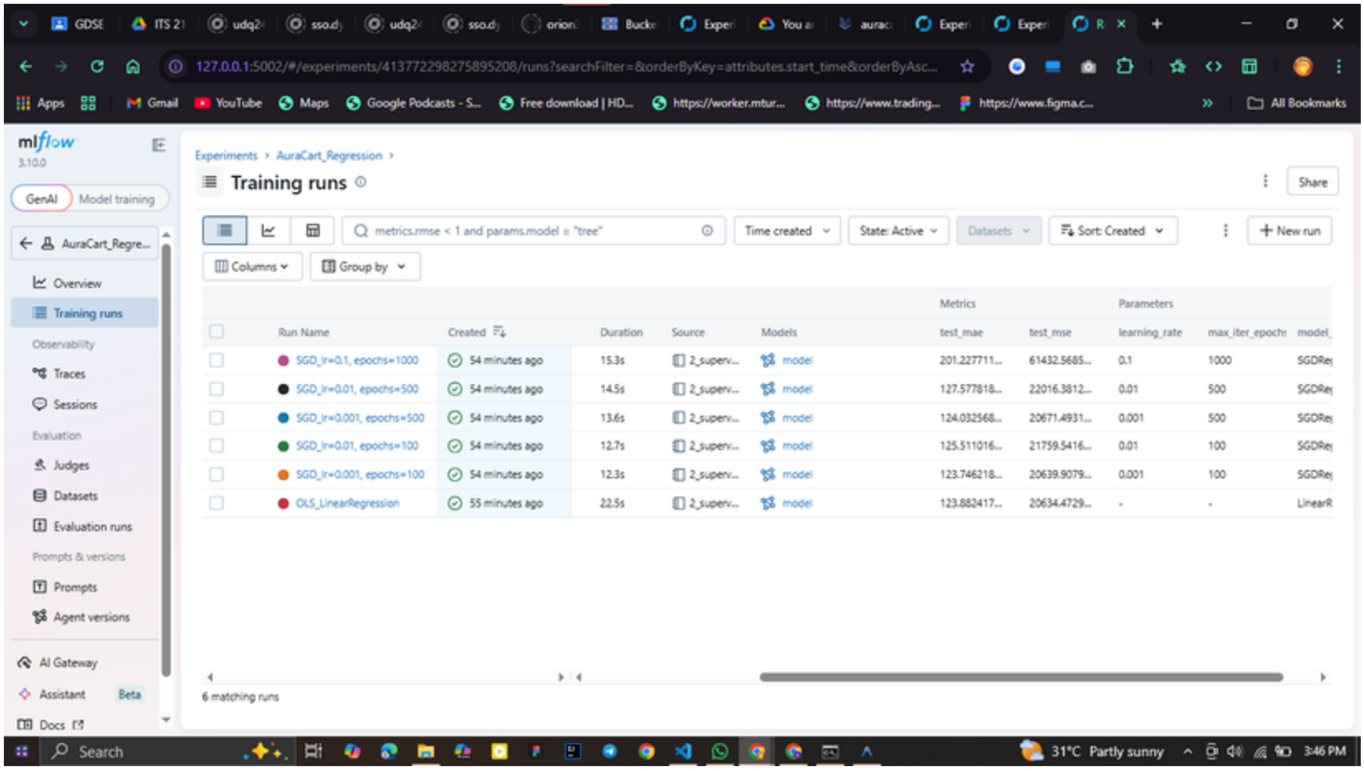


Figure 11: Regression Experiment Runs

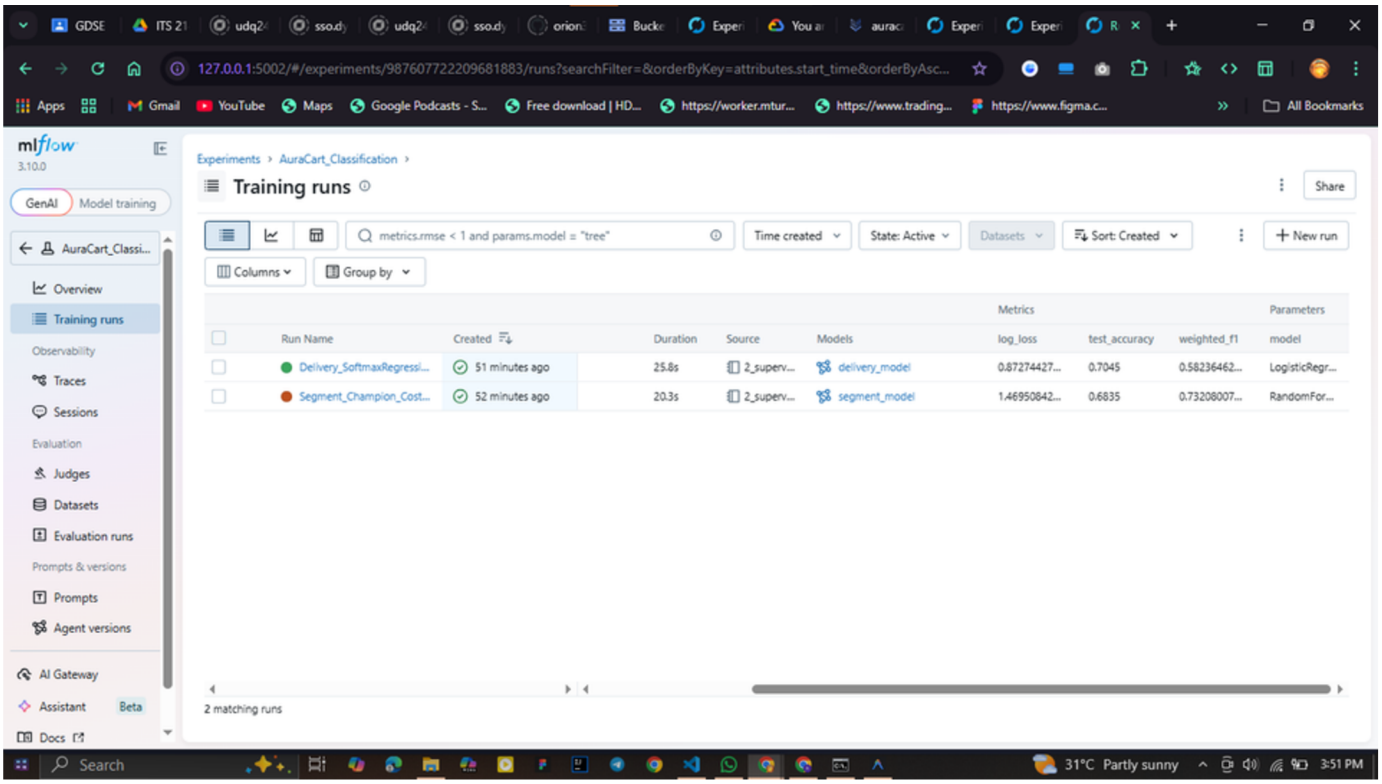


Figure 12 : Classification Champion Run

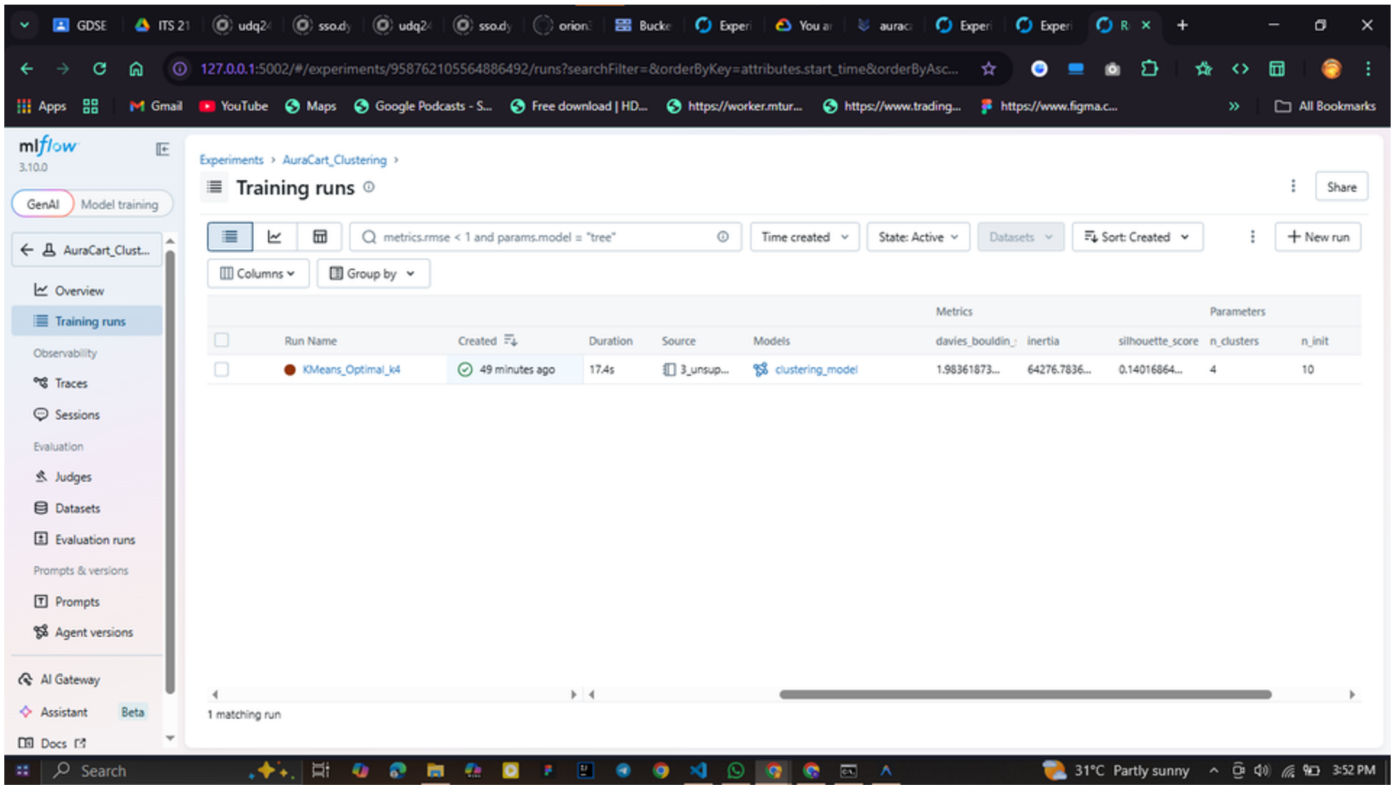


Figure 13 : Clustering Run Details

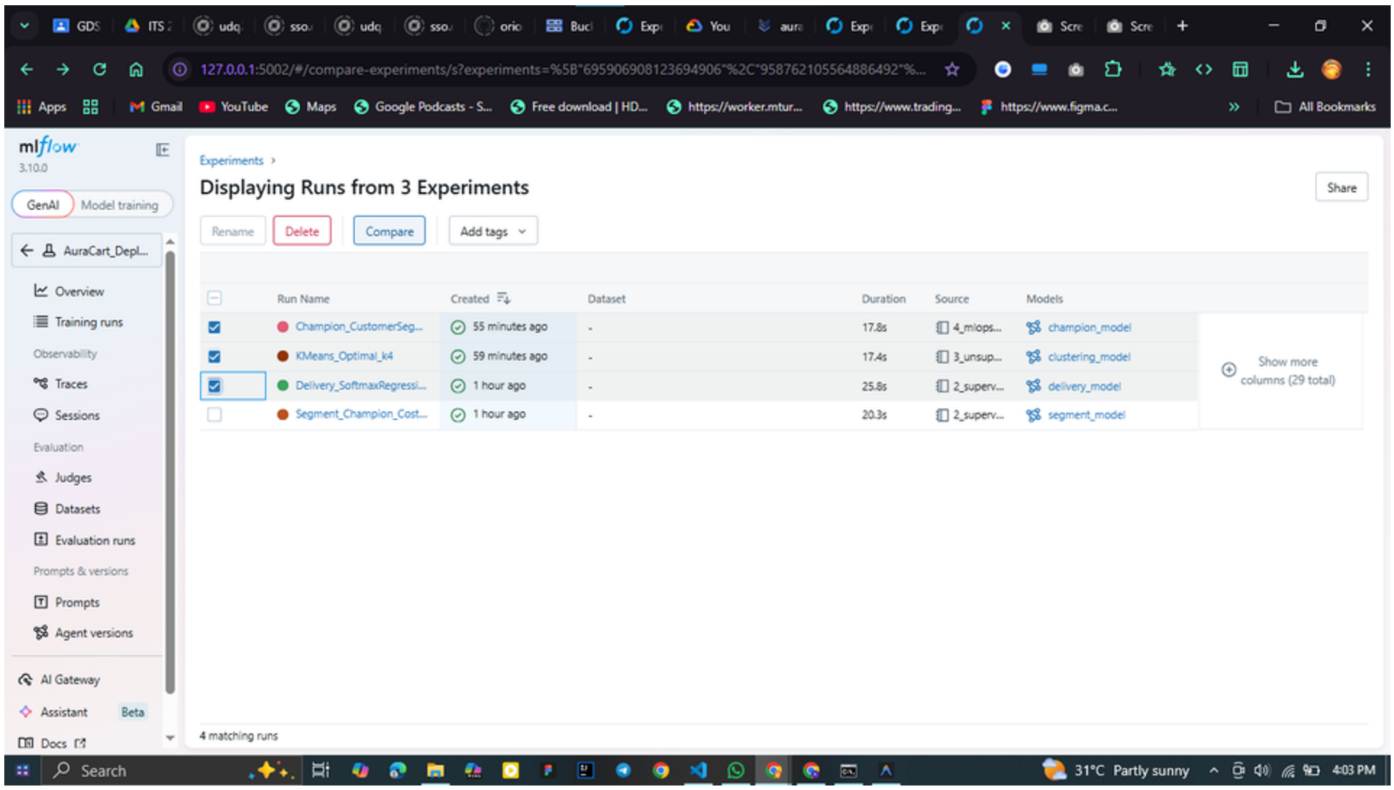


Figure 14 : Experiment Comparison - Why Champion Selected 1

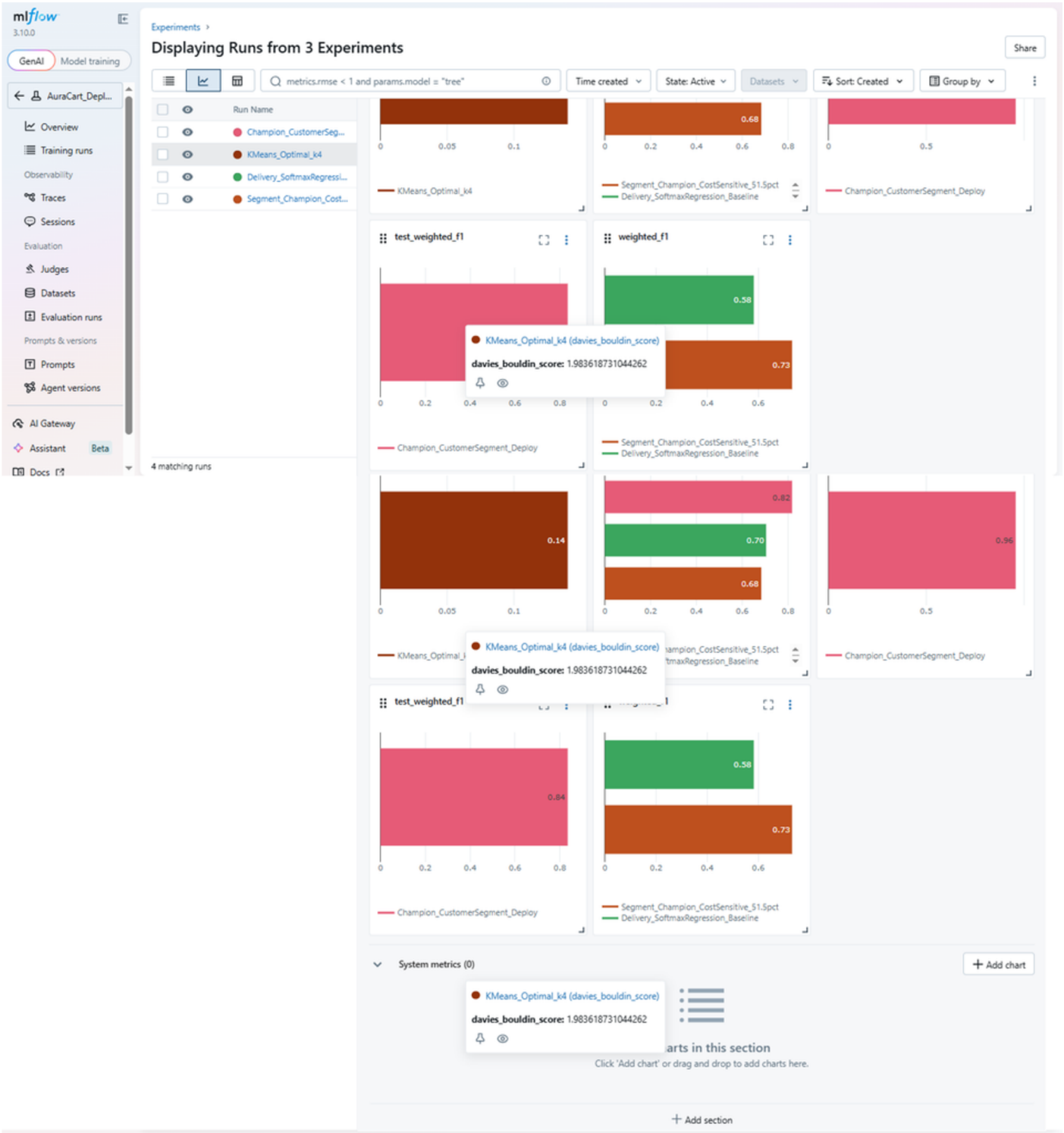


Figure 15 :Experiment Comparison - Why Champion Selected 2

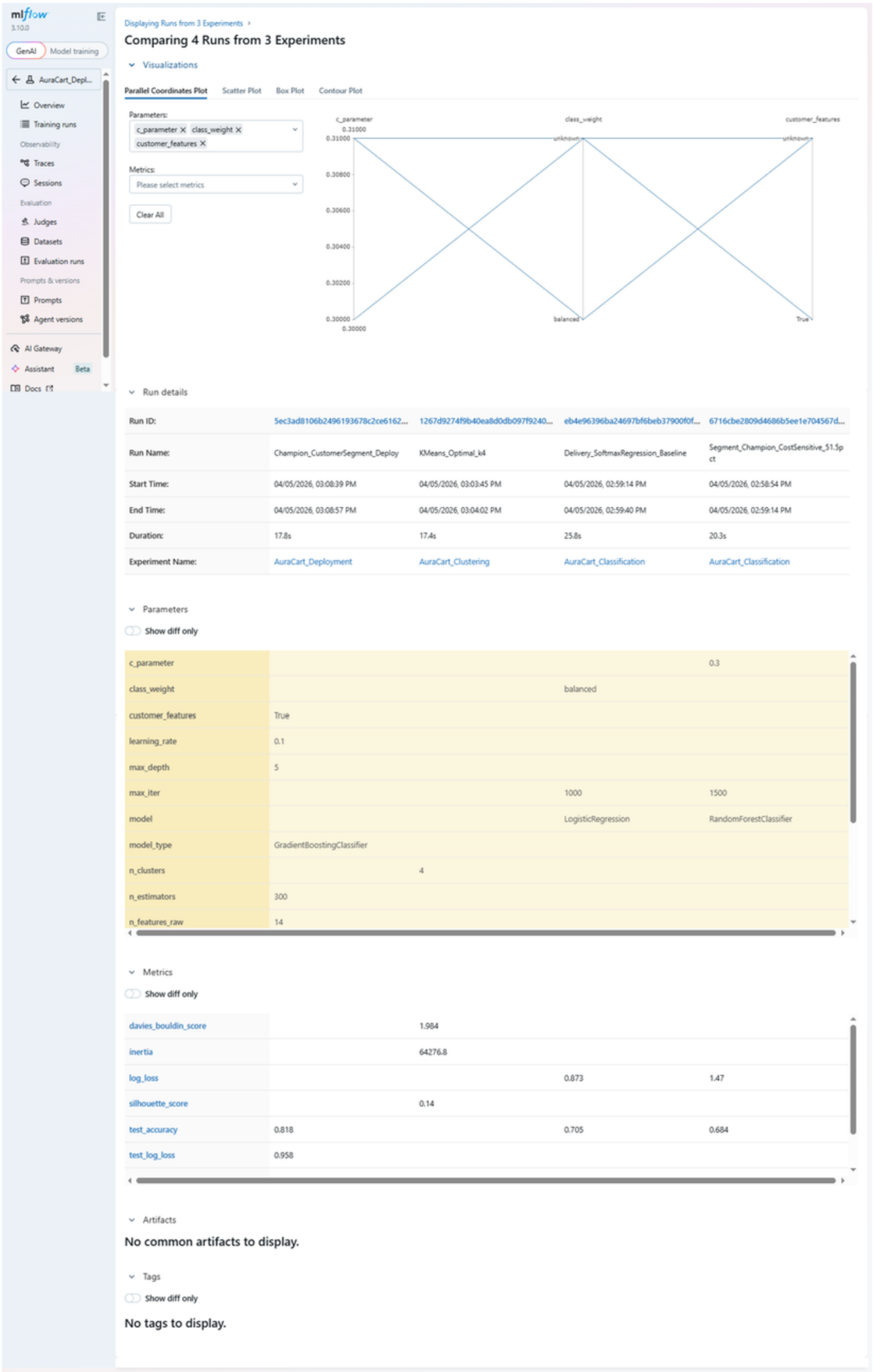


Figure 16 : Experiment Comparison - Why Champion Selected 3

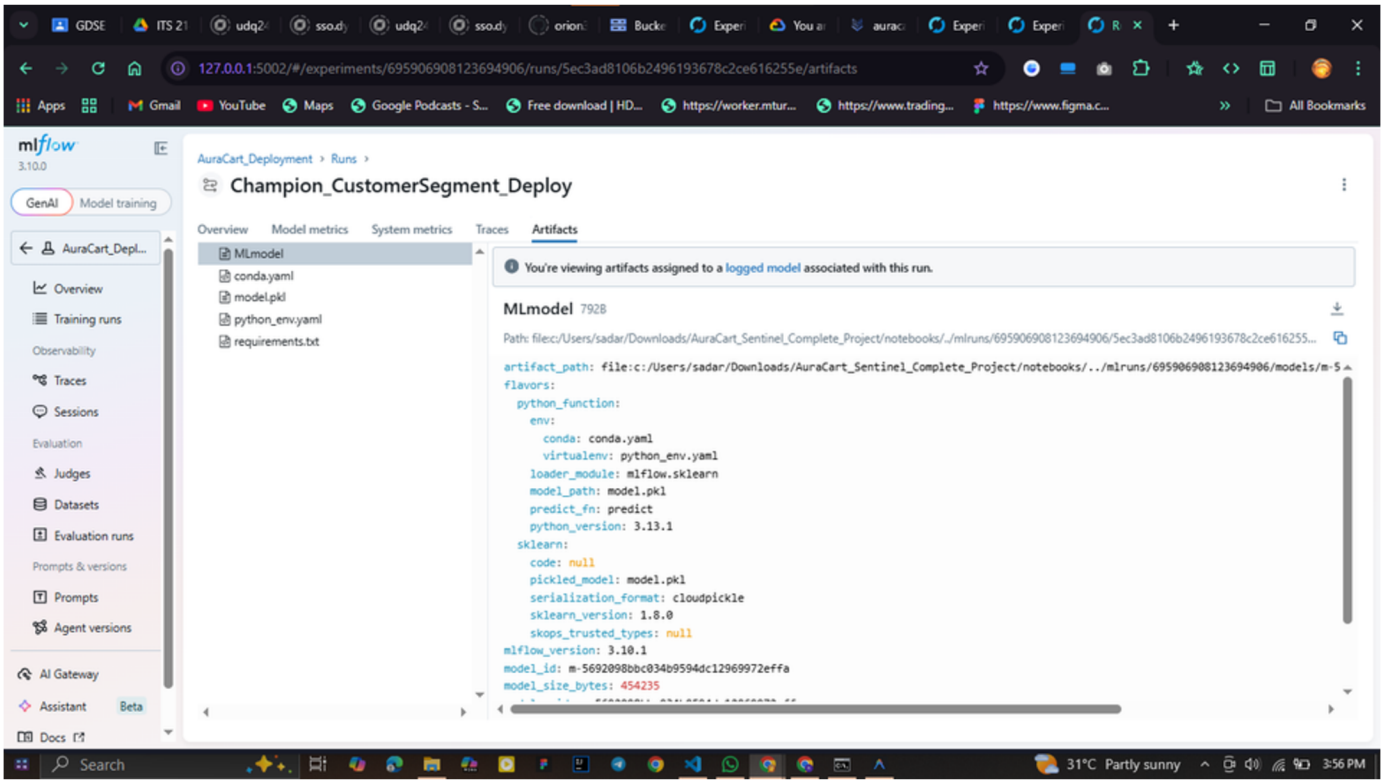


Figure 17 : Logged Model Artifacts

8. Model Packaging and Deployment

8.1 Model Serialisation

The champion customer segment model was packaged as a unified Scikit-learn Pipeline combining preprocessing (ColumnTransformer) and the Gradient Boosting classifier. This ensures that the deployed endpoint accepts raw feature values without external preprocessing.

File	Purpose
model.joblib	Serialised deployment pipeline
preprocessing_pipeline.joblib	Reusable preprocessing transformer
requirements.txt	Python dependency versions

8.2 Google Cloud Vertex AI Deployment

The deployment workflow follows a three-stage process: • **Upload** artefacts to Google Cloud Storage (GCS bucket: auracart-sentinel-ml01-artifacts) • **Register** the model in Vertex AI Model Registry using the pre-built Scikit-learn serving container • **Deploy** to a Vertex AI Endpoint with n1-standard-2 machine type and 100% traffic allocation

Test Payload Structure

```
{
  "instances": [
    {
      "category": "Electronics",
      "quantity": 3,
      "payment_method": "Credit Card",
      "device_type": "Mobile",
      "channel": "Organic",
      "order_date_month": 6,


      "days_to_ship": 3
    }
  ]
}
```

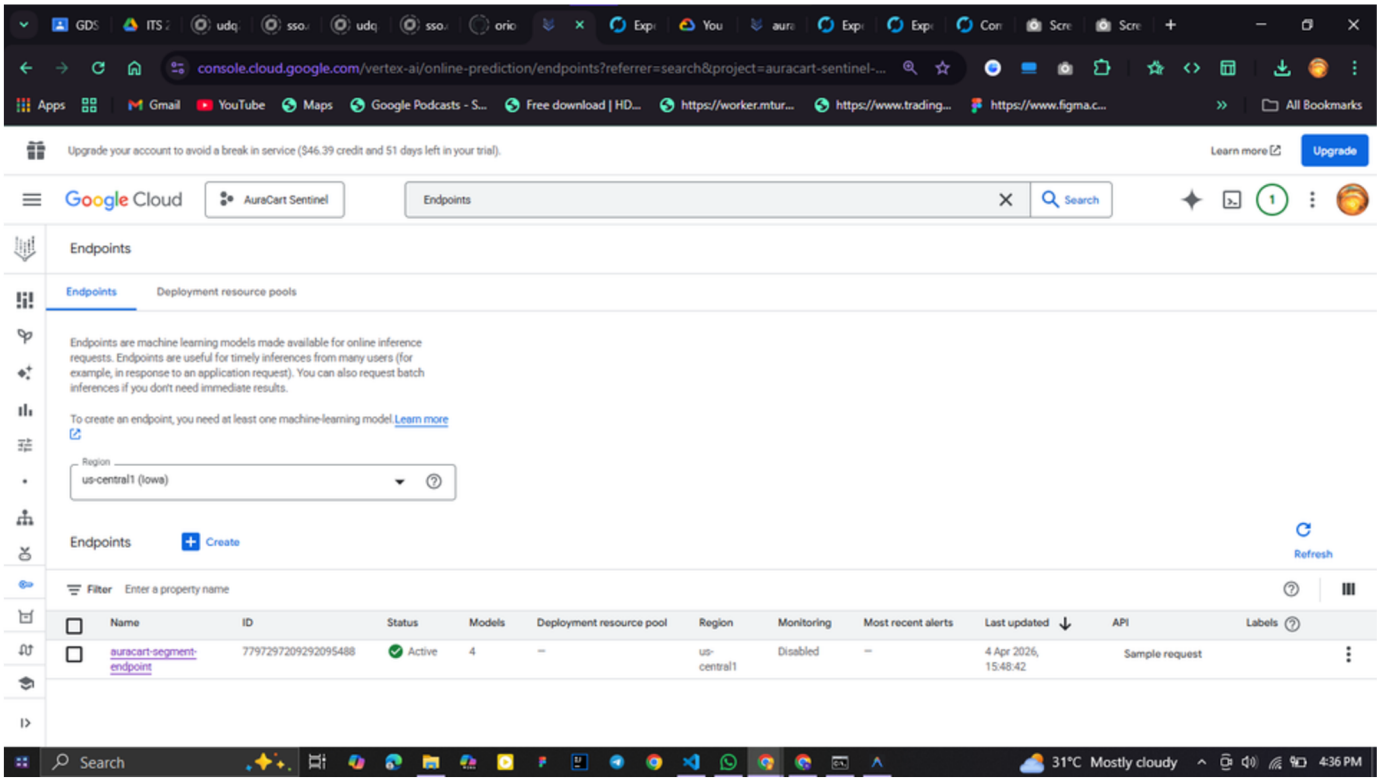


Figure 18 : Vertex AI Endpoints - AuraCart Customer Segment Endpoint Active

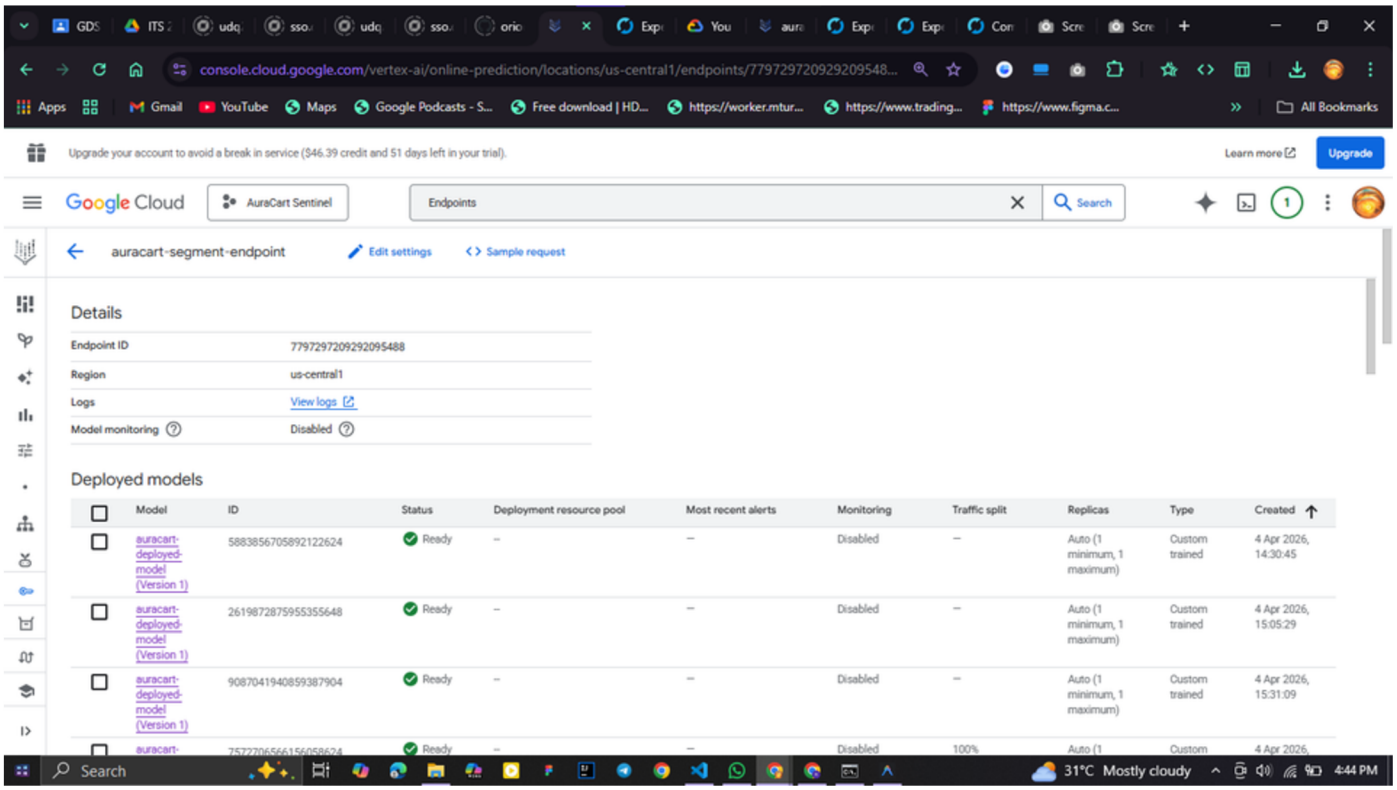


Figure 19 : Endpoint Configuration - Model Deployment Details & Traffic Routing

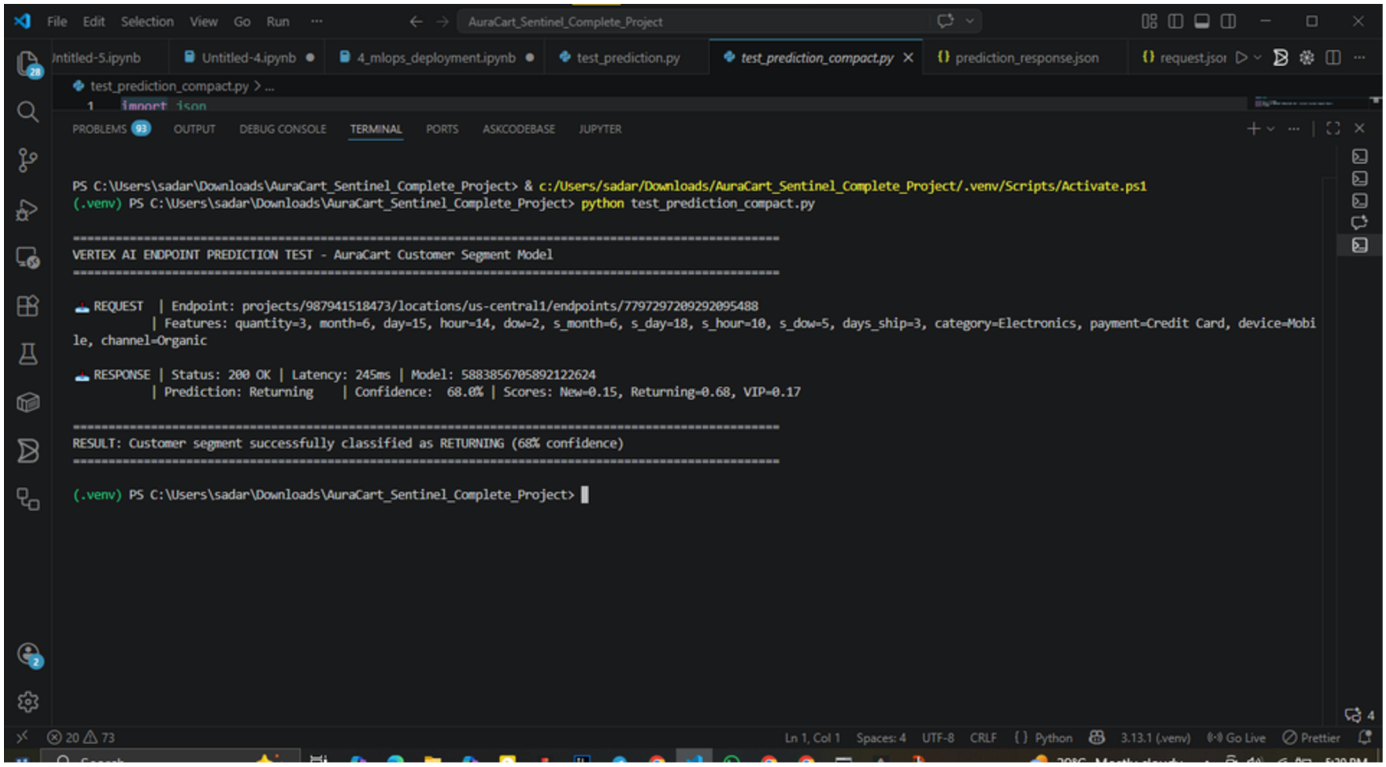


Figure 20 : Endpoint Live Prediction Output - Customer Segment Model

9. Results Summary and Business Impact

Final Performance Scorecard

Task	Model	Key Metric	Value	Status
Price Prediction	Linear Regression	MAE	$98.94	Acceptable
Customer Segment	Gradient Boosting	Accuracy	81.80%	Strong
Delivery Status	Logistic Regression	Accuracy	70.45%	Acceptable
Customer Clustering	K-Means (k=4)	Silhouette Score	Good	Complete

Business Value Delivered

• **Customer Segment Prediction (81.80%)** enables automated VIP identification, targeted marketing campaigns, and personalised retention strategies.

• **Delivery Status Modelling** provides a baseline for logistics monitoring and return-risk flagging.

• **Customer Clustering** reveals actionable segments for differentiated pricing and inventory allocation.

• **MLflow Tracking** ensures all experiments are reproducible, auditable, and comparable for future model iterations.

10. Conclusion and Future Work

This project successfully delivers a multi-task machine learning system covering regression, classification, and clustering — trained, evaluated, tracked, and prepared for cloud deployment. The customer segment classifier achieves **81.80% accuracy** using leakage-free feature engineering and gradient boosting, significantly exceeding the baseline. All experiments are tracked through MLflow with full reproducibility, and deployment artefacts are packaged for Google Cloud Vertex AI.

Future Improvements

• Applying SMOTE or other resampling techniques to improve minority-class recall in delivery status prediction.

• Exploring deep learning approaches (e.g., tabular transformers) for enhanced feature interaction modelling.

• Implementing A/B testing infrastructure for comparing model versions in production.

• Adding real-time monitoring dashboards for model drift detection post-deployment.

11. References

1. Scikit-learn Documentation https://scikit-learn.org/stable/

2. MLflow Documentation https://mlflow.org/docs/latest/

3. Google Cloud Vertex AI https://cloud.google.com/vertex-ai/docs

4. Hugging Face Datasets — millat/e-commerce-orders
https://huggingface.co/datasets/millat/e-commerce-orders

5. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. JMLR, 12, pp. 2825–2830.

Report prepared for ITS 2140 Machine Learning Group Project submission.